

D-NAT: Data-Driven Non-Ideality Aware Training Framework for Fabricated Computing-In-Memory Macros

Ming-Guang Lin^{ID}, Chi-Tse Huang^{ID}, Yu-Chuan Chuang^{ID}, Yi-Ta Chen^{ID}, *Student Member, IEEE*,
Ying-Tuan Hsu^{ID}, Yu-Kai Chen, Jyun-Jhe Chou^{ID}, Tsung-Te Liu^{ID}, *Member, IEEE*,
Chi-Sheng Shih^{ID}, *Senior Member, IEEE*, and An-Yeu Wu^{ID}, *Fellow, IEEE*

Abstract—To enable energy-efficient computation for deep neural networks (DNNs) at edge, computing-in-memory (CIM) is proposed to reduce the energy costs during intense off-chip memory access. However, CIM is prone to multiply-accumulate (MAC) errors due to non-idealities of memory crossbars and peripheral circuits, which severely degrade the accuracy of DNNs. In this work, we propose a Data-Driven Non-ideality Aware Training (D-NAT) framework to compensate for the accuracy degradation. The proposed D-NAT framework has the following contributions: 1) We measured a fabricated SRAM-based CIM macro to obtain a data-driven MAC error model (D-MAC-EM). Based on the derived D-MAC-EM, we analyze the impact of the non-idealities on DNN's accuracy. 2) To make DNNs robust to the non-idealities of CIM macros, we incorporate the measured D-MAC-EM into DNN's training procedure. Moreover, we propose a statistical training mechanism to better estimate the gradients of the discrete D-MAC-EM. 3) We investigate trade-offs between quantization range and quantization errors. To mitigate the quantization errors in activations, we introduce extended PACT (E-PACT) that adaptively learns the upper and lower bounds of input activations for each layer. Simulation results show that our proposed D-NAT improves the accuracy of ResNet20, VGG8, ResNet34, and VGG16 by 78.98%, 71.8%, 72.04%, and 57.85%, respectively, which reaches the ideal upper bound of the quantized model. Lastly, the D-NAT framework is validated on an FPGA platform with the fabricated SRAM-based CIM macro chip. Based on the measurement results, D-NAT successfully recovers the accuracy under non-idealities of a real SRAM-based CIM macro.

Index Terms—Computing-in-memory (CIM), deep neural network (DNN), non-ideality aware training.

I. INTRODUCTION

DEEP neural networks (DNNs) have demonstrated overwhelming success in various machine learning applications, including image processing [2], language translation [3], and speech recognition [4]. However, intense data movements between computational units and off-chip memories pose a significant challenge to conventional accelerators based on the von-Neumann architecture. Recently, computing-in-memory (CIM) [5], [6] mechanism has been proposed as a promising solution to the von-Neumann bottleneck through the integration of computational units and memories. CIM-based accelerators can parallelly perform multiply-accumulate (MAC) operations, the dominant operation in DNNs, in the analog domain within a memory crossbar. Therefore, the energy efficiency is significantly improved by minimizing the costs of data movements.

However, CIM-based accelerators suffer from the reliability issue of analog computation [7]–[12]. Since outputs of the memory crossbar are sensed as a summation of currents through memory cells, following Kirchhoff's current law, they are prone to MAC errors due to the non-ideal behavior of the memory crossbar. Moreover, peripheral circuits such as digital-to-analog converters (DACs) and analog-to-digital converters (ADCs) also cause deviations in the computed values and severely degrade the accuracy of DNN.

Many works have been proposed to address the reliability issue of CIM. After our examination, they can be categorized as 1) Hardware-based calibration methods, and 2) Analytical non-ideality aware training (A-NAT) methods:

- 1) In the hardware-based calibration methods, dedicated hardware is responsible for compensating for the non-idealities. In [13], the authors proposed a compact row-by-row ADC with replica bitcells for mitigating the non-linearity and process variations. In [16], a customized metal-oxide-metal (MOM) capacitor was applied to minimize the variations of the stored data. The body bias tuning technique for PMOS/NMOS mismatch

Manuscript received December 24, 2021; revised February 22, 2022 and April 21, 2022; accepted April 26, 2022. Date of publication April 29, 2022; date of current version June 13, 2022. This work was supported in part by the Ministry of Science and Technology of Taiwan under Grant MOST 110-2218-E-002-034-MBK and Grant MOST 111-2218-E-002-018-MBK; and in part by MediaTek Inc., under Contract MTKC-2021-0167. This article was recommended by Guest Editor J.-Y. Kim. (Ming-Guang Lin and Chi-Tse Huang contributed equally to this work.) (Corresponding author: An-Yeu Wu.)

Ming-Guang Lin, Chi-Tse Huang, Yu-Chuan Chuang, Yi-Ta Chen, Ying-Tuan Hsu, Yu-Kai Chen, and Tsung-Te Liu are with the Graduate Institute of Electronics Engineering, National Taiwan University, Taipei 10617, Taiwan.

Jyun-Jhe Chou is with the Department of Computer Science and Information Engineering, National Taiwan University, Taipei 10617, Taiwan.

Chi-Sheng Shih is with the Department of Computer Science, National Taiwan University, Taipei 10617, Taiwan.

An-Yeu Wu is with the Department of Electrical Engineering, National Taiwan University, Taipei 10617, Taiwan (e-mail: andywu@ntu.edu.tw).

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/JETCAS.2022.3171268>.

Digital Object Identifier 10.1109/JETCAS.2022.3171268

2156-3357 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

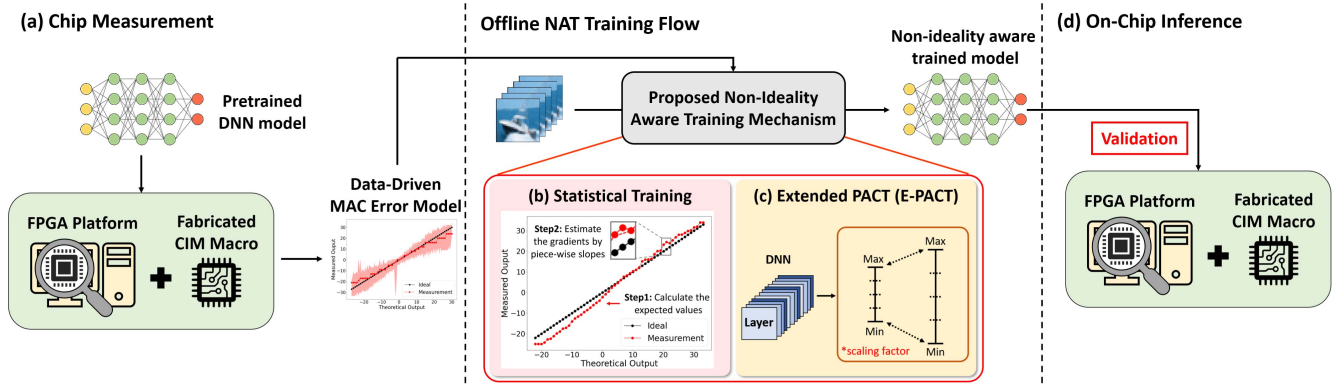


Fig. 1. The overall data-driven non-ideality aware training (D-NAT) framework, including (a) data-driven modeling from real chip measurement, (b) statistical training mechanism, (c) extended PACT (E-PACT), and (d) on-chip inference for validation.

and the reference voltage calibration was applied for the local variability of the ADC in [14]. The authors of [15] used 8T static random-access memory (SRAM), instead of the traditional 6T SRAM, to provide sufficient noise margin. Although the hardware-based methods can mitigate the non-idealities, the additional circuit components impose area overhead. Moreover, they do not consider the statistical characteristics of data converters and their impacts on DNN's accuracy.

- 2) On the other hand, A-NAT methods utilized the intrinsic resilience of DNN models to make them robust to the non-idealities during the training process. Stochastic data-driven hardware resilience (S-DDHR) [19] used Gaussian distribution to analytically model hardware variability of magnetoresistive random-access memory (MRAM)-based CIM. Afterwards, S-DDHR applied stochastic training to DNNs with the analytical variation model. In [20], an iterative training algorithm was proposed to regularize model parameters to fit into a linear operating range of 1T1R cells in a layer-by-layer manner to compensate for the accuracy degradation. CxDNN [18] proposed a software training flow for non-ideal resistive random-access memories (ReRAMs) by modeling the non-idealities of ReRAM cells and DACs/ADCs with an analytical variation model. These A-NAT methods show promising results in mitigating the effect of process variations generated by MRAM/ReRAM-based CIM. However, most variation models are based on “pre-defined” analytical models without validation on a real fabricated chip. Moreover, none of them apply their methods to SRAM-based CIM macros.

To solve the above issues, this paper establishes a data-driven non-ideality aware training (D-NAT) framework for non-ideal CIM macros. “Data-driven” indicates that D-NAT utilizes the measured data that has been influenced by the non-idealities of the CIM macro to train an error-aware model. The overall D-NAT framework is depicted in Fig. 1. The main contributions of this work are as follows:

- 1) **Investigation of the major differences between the pre-defined A-MAC-EM and the practical D-MAC-EM:** We set up an FPGA platform to measure a fabricated SRAM-based CIM macro [1]. By comparing

theoretical outputs with measured outputs, we obtain a data-driven MAC error model (D-MAC-EM) for the following statistical training process. Furthermore, we investigate the difference between the measured D-MAC-EM and a conventional analytical MAC error model (A-MAC-EM) with simulated variations. The experimental results indicate that A-MAC-EM is insufficient to represent the non-idealities of a practical SRAM-based CIM macro, highlighting the importance of validation on a real fabricated chip.

- 2) **Statistical training mechanism:** Since D-MAC-EM is a sequence of discrete points with a probabilistic distribution, we propose a statistical training mechanism to better estimate the gradient of each point. Specifically, we first compute the expected value for each point with the probabilistic distribution. The gradient of each point is estimated as the slope of the piece-wise linear curve between two adjacent points. Compared with straight-through-estimator (STE) [24], which does not consider the statistical characteristics of the measured D-MAC-EM, the proposed statistical training mechanism improves the training speed and the accuracy of ResNet20 and VGG8 by 0.55% and 1.3%, respectively.
- 3) **Extended PACT (E-PACT):** We first investigate the trade-offs between quantization range and quantization errors for a quantized model. Based on the observation, we introduce E-PACT to automatically determine a quantization range for each layer. E-PACT adaptively learns the quantization range by parameterizing the upper bounds and the lower bounds of input activations. Compared with clipped ReLU [3], E-PACT clips the input activations within a more proper range by considering the information of negative values. Experimental results show that E-PACT improves the accuracy of ResNet20 and VGG8 by 1.1% ~ 30.1% and 0.51% ~ 12.06%, respectively.
- 4) **Validation of D-NAT trained model on a fabricated CIM Macro:** We validate the proposed D-NAT framework on the FPGA platform with the fabricated SRAM-based CIM macro [1] for on-chip inference. The measurement results show that the accuracy of VGG8 is enhanced from 14.17% to 87.17% without on-chip calibration. Overall, D-NAT shows its superiority by

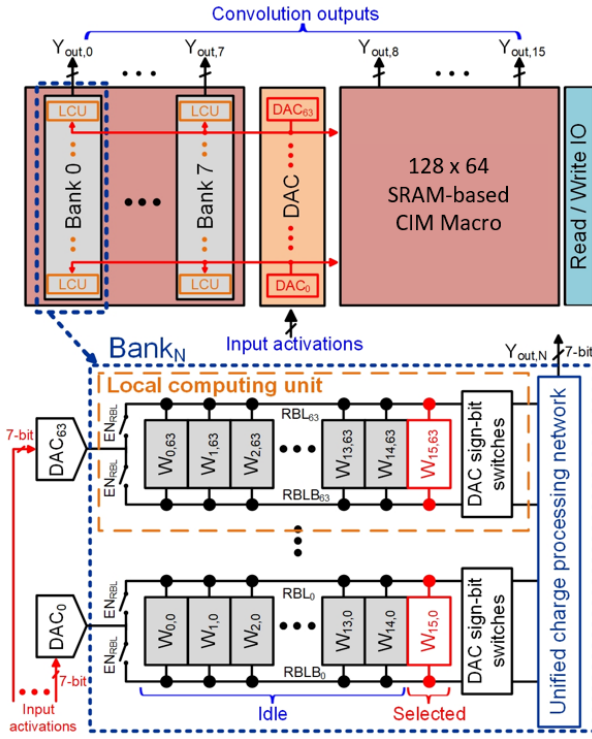


Fig. 2. Macro architecture of the SRAM-based CIM [1].

considering the non-idealities and quantization errors of a CIM macro during the training process without additional hardware components. Furthermore, the proposed D-NAT framework can be generalized to other SRAM-based CIM macros by training DNNs with the measured D-MAC-EM.

The rest of this paper is organized as follows. Section II introduces the SRAM-based CIM macro used in this work and the current status of A-NAT. Section III presents the measurement results of non-idealities of the fabricated SRAM-based CIM macro. The proposed D-NAT framework is presented in Section IV. Section V and Section VI shows the simulation results and the validation results on the CIM macro, respectively. Finally, we conclude this paper in Section VII.

II. PRELIMINARIES

A. Fabricated SRAM-Based CIM Macro Used in This Work [1]

In [1], the authors implemented an SRAM-based CIM macro with a unified charge-processing network (UCPN), and the proposed D-NAT will be derived based on our observations and measurements of this fabricated CIM macro. Fig. 2 shows the overall architecture of the CIM macro, which includes 64 7-bit dynamic current-steering DACs, 16 computation banks, and a UCPN that offers the capability of data conversion equivalent to 16 7-bit ADCs with a maximum filter depth of 64. Each computation bank consists of 64 local computing units (LCU) responsible for weight storing and MAC operations.

During computation, DACs first convert 7-bit digital input activations into corresponding analog voltages for pre-charging a computation bank. The multiplication operations are performed between voltages on bit lines and weights in SRAM

cells by enabling the word line. Finally, the UCPN accumulates the multiplication results and converts the analog signals back to digitized outputs.

The SRAM-based CIM macro can conduct the convolutions with 64 7-bit activation inputs and 64 1-bit weights in one bank and collect one accumulated output per cycle. The SRAM-based CIM macro includes 16 banks to conduct the computations in parallel with input activations sharing between the computation banks. The equation of the MAC operation in the SRAM-based CIM macro is shown as below:

$$Y_{out,N} = \frac{1}{64} \sum_{i=0}^{63} X_i \times W_{f,i}, \quad (1)$$

where $Y_{out,N}$ represents the digital output of bank N , X_i is the i^{th} activation input for each bank, and $W_{f,i}$ represents the i^{th} weight stored in the f^{th} column in each bank.

B. Analytical Non-Ideality Aware Training (A-NAT) Flow

In this part, we summarize the methodology of conventional A-NAT. For ReRAM-based CIM macros, A-NAT first analytically models the non-idealities such as wire resistance and imperfect write variations. The software training flow consists of the following three steps: quantization, conversion, and retraining. During the quantization phase, A-NAT quantizes weights and activations into fixed-point numbers, which matches the precision of the ReRAM-based CIM macro. The quantized model is mapped to the memory crossbars in the conversion phase, which converts the activations to voltage and weights to conductance. In the retraining phase, A-NAT forward propagates input data with the analytical model to obtain MAC errors generated from the non-idealities. After that, backward propagation, gradient computation, and weight updating are applied to the floating-point model to compensate for the non-idealities.

While A-NAT achieves accuracy improvements under the non-ideal ReRAM-based CIM, the following issues remain open:

- 1) **Modeling practical non-ideal effects with analytical models:** During the training process, A-NAT uses a gaussian distribution to analytically model the device-level non-idealities. However, modeling all the variations in the CIM macro is “impractical”. On the other hand, the analytical model is not consistent with a practical CIM circuit. Therefore, the non-ideality-aware trained DNN may still suffer from accuracy degradation while deploying on a real fabricated chip.
- 2) **Estimating the gradients of the MAC error model with an identity function:** Quantization-aware training (QAT) [21] is proposed to enhance accuracy by considering quantization errors during the training process. To compute the gradients of the discrete quantized function, STE [24] is widely used to estimate the gradients with an identity function. However, the quantization function is not linear considering the non-ideal effects of CIM. The mismatch of the gradients and the forward function degrades DNN’s accuracy and reduces the convergence speed of the training process.

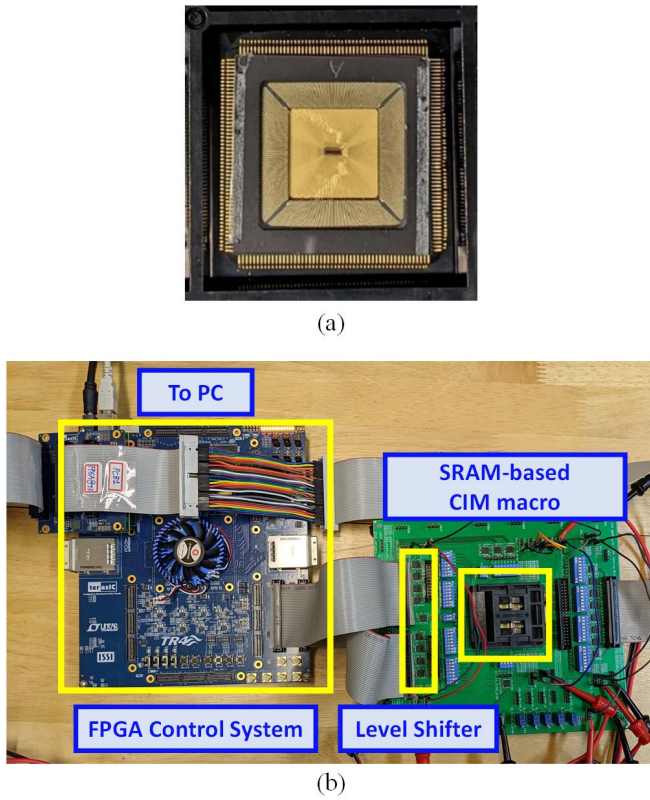


Fig. 3. (a) The package photo of the fabricated SRAM-based CIM macro chip in [1], and (b) the FPGA platform for chip measurement.

- 3) **Determining the quantization range for input activations in a heuristic way:** The quantization scheme for input activations determines the trade-offs between quantization range and quantization errors. A-NAT exhaustively tries all possible candidates of quantization ranges in a pre-set list and selects the one with the highest classification accuracy in a layer-wise manner. This heuristic solution may be suboptimal and causes high computation time during the training process.

In this work, we propose a Data-Driven Non-Ideality Aware Training (D-NAT) framework to address the above issues while considering the real measurement data from a fabricated SRAM-based CIM macro. The details will be illustrated in the following sections.

III. OBSERVATIONS OF DATA-DRIVEN NON-IDEALITIES FROM A FABRICATED SRAM-BASED CIM MACRO

This section presents the measurement results of the fabricated SRAM-based CIM macro [1] to obtain its real D-MAC-EM. Furthermore, we depict several observations of the non-ideal effects based on the D-MAC-EM. The package photo of the chip and the measurement environment are shown in Fig. 3.

A. Strategy for Mapping DNNs Onto SRAM-Based CIM Macro

In this work, VGG8 and the CIFAR-10 dataset [27] are used to demonstrate the functionality of the SRAM-based CIM

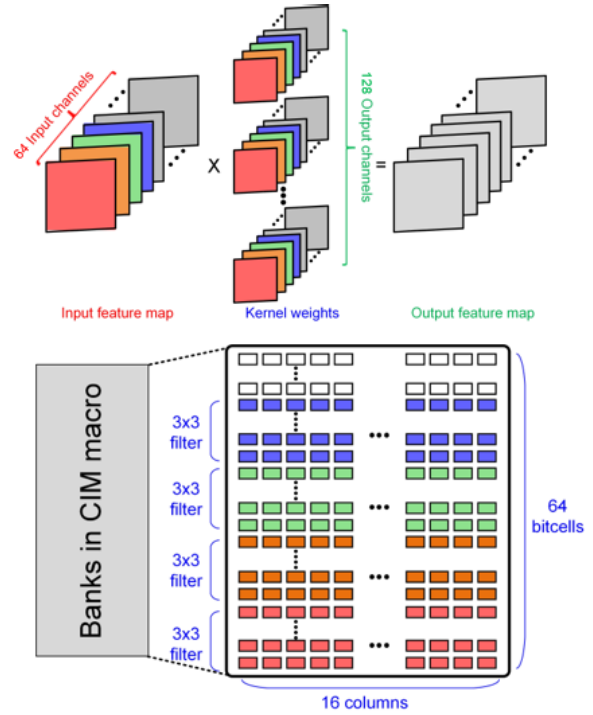


Fig. 4. The mapping example of the convolution layer 2 to the SRAM-based CIM macro.

macro. The network structure of VGG8 includes 5 convolution layers and 3 fully connected layer. Each convolution layer is followed by a rectified linear unit (ReLU) layer and a batch normalization layer. Specifically, only the middle 4 convolution layers are computed in the SRAM-based CIM macro.

The mapping example of the convolution layer 2 to the SRAM-based CIM is shown in Fig. 4. The convolution layer 2 has 64 input channels and 128 output channels, and its filter size is 3×3 . In the SRAM-based CIM macro, each bank has 16 columns to store the weights, each column has 64 bitcells. Hence, 8 banks are required for computing the convolution layer 2 with 128 output channels. In this work, four channels in the same kernel are allocated on the same column, and the remained bitcells are stored with data “0”. Therefore, 16 times of re-loading weights into the banks are required to compute kernels with 64 channels. During the computation, the SRAM-based CIM macro needs 16 cycles to switch 16 weight columns in each bank to compute each partial sum. The 8 banks in the SRAM-based CIM macro can operate parallelly to generate the partial sums of the output feature map.

B. Data-Driven MAC Error Model (D-MAC-EM)

The D-MAC-EM of the SRAM-based CIM macro is analyzed and obtained by measuring the MAC computation results of VGG8 on CIFAR-10. After comparing the measured outputs to the theoretical outputs, the D-MAC-EM is shown in Fig. 5(a). For each theoretical output, the measured outputs are calculated and listed as the measured curve. The solid black line in the figure represents the ideal curve, which is a linear and one-to-one mapping. The red region includes all possible

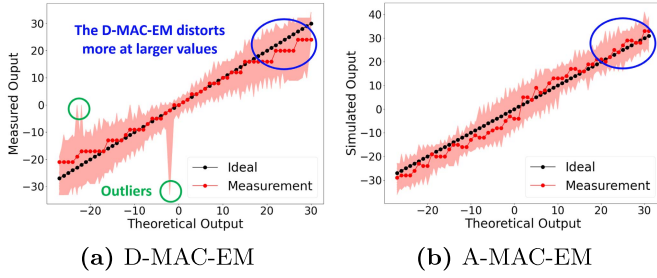


Fig. 5. (a) The D-MAC-EM obtained by measuring a real SRAM-based CIM macro [1]. Note that the measured output distorts more at larger values. (b) The A-MAC-EM obtained by adding Gaussian noises, which is quite uniform and cannot reflect the practical non-ideal effects in the fabricated macro.

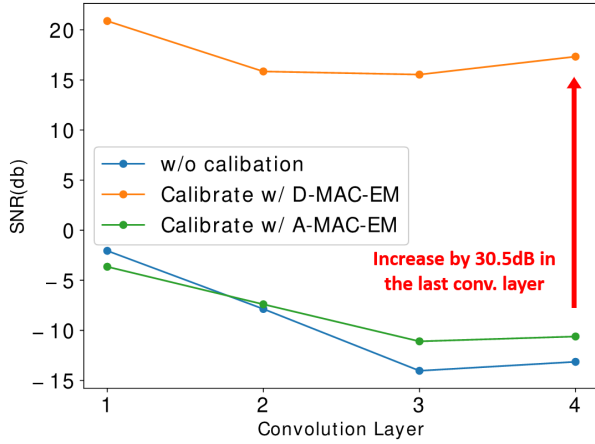


Fig. 6. Comparison of the SNR results of computed outputs under different experimental setups.

measured outputs, which is a discrete probability distribution due to the data-dependent MAC computation. The solid red line forms a one-to-one mapping from each theoretical output to measured output with the maximum probability.

On the other hand, to develop the A-MAC-EM for the target SRAM-based CIM macro, we assume the variation mainly occurs in ADC and DAC [28]. To generate our A-MAC-EM and analyze the non-idealities in ADC and DAC, we modify the behavior model of SAR-ADC in Matlab [29], [30] by the following three steps:

1. Add the Gaussian noises to simulate the DAC input variations.
2. Adjust the weights of capacitances in ADC to model the capacitance mismatch from the process variation.
3. Inject the second-order harmonics to simulate the input signal symmetry, as well as the sample-and-hold non-linearity for ADC.

Fig. 5(b) shows the theoretical outputs and their corresponding simulated outputs of the A-MAC-EM, obtained by adding gaussian noises to input activations. Comparing the two error models, we observe the following differences: 1) The distribution of the D-MAC-EM has a larger variance at larger values, while that of the A-MAC-EM has a uniform variance at each value. 2) The measured outputs of the D-MAC-EM distort more and present a ladder-like distribution at larger values. 3) Some outlier points in the D-MAC-EM do

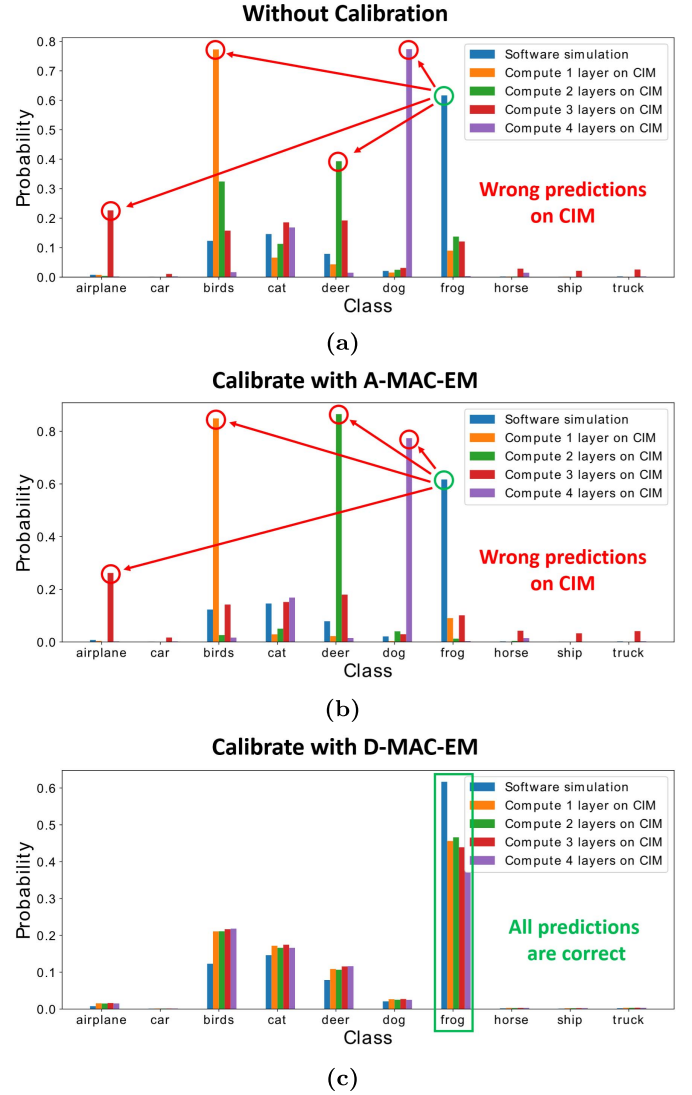


Fig. 7. Illustration of the prediction results for a frog image on the SRAM-based CIM macro under the following conditions: (a) without calibration, (b) calibrate with the conventional A-MAC-EM, and (c) calibrate with the measured D-MAC-EM.

not appear in the A-MAC-EM. The observation shows that the A-MAC-EM fails to model the practical situation.

C. Analysis of SNR Results

To evaluate the difference between D-MAC-EM and A-MAC-EM, we calculate signal-to-noise ratios (SNR) of the computed outputs, which are distorted by the non-ideal CIM, under the following situations: 1) without calibration, 2) calibrate with the A-MAC-EM, and 3) calibrate with the D-MAC-EM. During the calibration process, lookup tables are required to conduct one-to-one mappings for measured outputs. Since the measured outputs of both error models are with probability distributions, we choose the measured output with the maximum probability to construct corresponding lookup tables for both error models.

The SNR results are shown in Fig. 6. The blue line shows the SNR results between the measured and theoretical

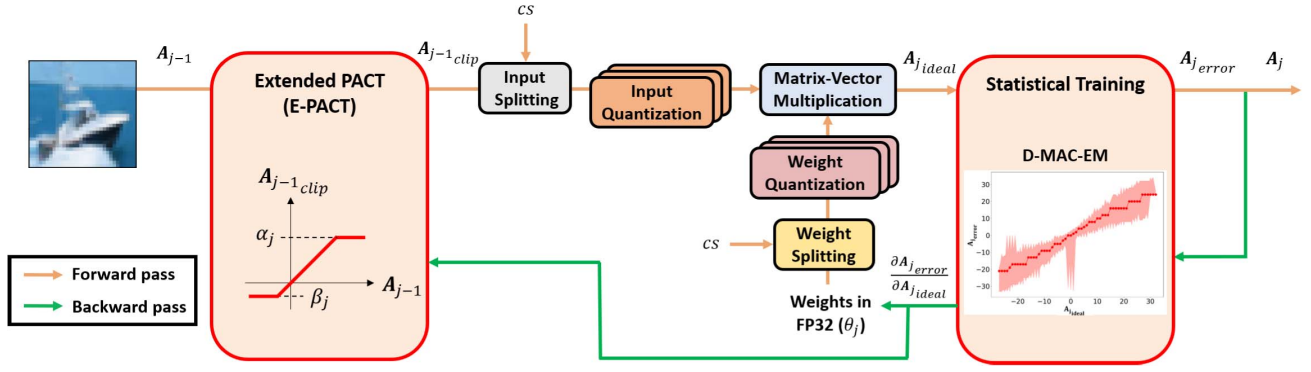


Fig. 8. The computation flow of our proposed non-ideality aware training (NAT) mechanism in the j^{th} layer.

outputs without calibration. The green line is the SNR results calibrated by the A-MAC-EM, and the orange line is the SNR results calibrated by the D-MAC-EM. The SNR of the computed outputs can be enhanced by 30.47dB in the last convolution layer of VGG8 when considering the D-MAC-EM. On the other hand, the SNR results of the A-MAC-EM only improve 2.53dB compared with that without calibration, indicating that the A-MAC-EM is insufficient to represent the actual non-idealities in the SRAM-based CIM macro.

D. Prediction Results of One Image

We further analyze the prediction results of inferencing one image of the frog class in the CIFAR-10 dataset on VGG8. Fig. 7 illustrates the probability of the 10 classes. The ideal case means we compute all the convolution layers in the digital domain to ensure no computation errors in results. The layer n means we compute the first n convolution layers in the SRAM-based CIM macro, and the remaining layers are computed in the digital domain.

Fig. 7(a) shows the prediction results without calibration, which fails to predict the correct class after computing the first convolution layer in the SRAM-based CIM macro. Fig. 7(b) shows the results calibrated with A-MAC-EM, which also fails to predict after computing the first convolution layer in the SRAM-based CIM macro. On the other hand, Fig. 7(c) shows the results calibrated with the D-MAC-EM. It successfully predicts the frog class when computing all the convolution layers in the SRAM-based CIM macro.

Based on the observations, despite the fact that prior A-MAC-EMs are effective in their works, they fail in the practical situation. The main reason is that they only verify their methods through software simulations with the testing environment constructed similarly to the training flow. However, it is impossible to completely model all the factors in the practical situation. Therefore, applying the A-MAC-EM to the practical testing data (measured from the fabricated chip) is not effective in this work. In conclusion, we have shown that the traditional A-MAC-EM is not enough for correcting the MAC computation errors generated from the SRAM-based CIM macro. Instead, the D-MAC-EM is more coherent to the non-idealities of the SRAM-based CIM macro and is required for correct prediction.

Algorithm 1 Non-Ideality Aware Training (NAT) Mechanism

Require:

model parameters θ_j , upper bound α_j , lower bound β_j , input activations A_{j-1} ($x_{train} = A_0$), forward function $f_j(\cdot)$, learning rate γ , training labels y , number of layers L , data-driven MAC error model Err , memory crossbar size cs

Ensure:

1: Forward Propagation

2: for $j = 1$ to L do

3: $A_{j-1}^{clip} = E-PACT(A_{j-1}, \alpha_j, \beta_j)$

4: $A_{j}^{ideal}, sf_{weight}, sf_{activation} = f_j(A_{j-1}^{clip}, \theta_j, cs)$

5: $A_{j}^{error} = Err(A_{j}^{ideal})$

6: $A_j = A_{j}^{error} \times sf_{weight} \times sf_{activation}$

7: end for

8: $Loss = Cross_Entropy(A_L, y)$

9: Backward Propagation

10: Compute $\frac{\partial A_{j}^{error}}{\partial A_{j}^{ideal}}$ //gradients for Statistical Training

11: Compute $\frac{\partial A_{j-1}^{clip}}{\partial \alpha_j}, \frac{\partial A_{j-1}^{clip}}{\partial \beta_j}$ //gradients for E-PACT

12: for $j = L$ to 1 do

13: $\frac{\partial Loss}{\partial \theta_j} = \frac{\partial Loss}{\partial A_{j}^{error}} \times \frac{\partial A_{j}^{error}}{\partial A_{j}^{ideal}} \times \frac{\partial A_{j}^{ideal}}{\partial \theta_j}$

14: $\frac{\partial Loss}{\partial \alpha_j} = \frac{\partial Loss}{\partial A_{j}^{error}} \times \frac{\partial A_{j}^{error}}{\partial A_{j}^{ideal}} \times \frac{\partial A_{j}^{ideal}}{\partial A_{j-1}^{clip}} \times \frac{\partial A_{j-1}^{clip}}{\partial \alpha_j}$

15: $\frac{\partial Loss}{\partial \beta_j} = \frac{\partial Loss}{\partial A_{j}^{error}} \times \frac{\partial A_{j}^{error}}{\partial A_{j}^{ideal}} \times \frac{\partial A_{j}^{ideal}}{\partial A_{j-1}^{clip}} \times \frac{\partial A_{j-1}^{clip}}{\partial \beta_j}$

16: end for

17: for $j = 1$ to L do

18: $\theta_j \leftarrow \theta_j - \gamma \times \frac{\partial Loss}{\partial \theta_j}$

19: $\alpha_j \leftarrow \alpha_j - \gamma \times \frac{\partial Loss}{\partial \alpha_j}$

20: $\beta_j \leftarrow \beta_j - \gamma \times \frac{\partial Loss}{\partial \beta_j}$

21: end for

IV. PROPOSED NON-IDEALITY AWARE TRAINING MECHANISM

In this section, we present the non-ideality aware training (NAT) mechanism based on the D-MAC-EM obtained in Section III. The framework consists of two techniques to mitigate the non-ideal effects of CIM: 1) statistical training

mechanism and 2) extended PACT (E-PACT). Statistical training aims to directly learn the D-MAC-EM into DNN model parameters. Therefore, extra calibration circuits are not required when performing on-chip inference. E-PACT automatically clips the input activations within a proper range to reduce the accuracy degradation owing to the quantization error.

A. Overview of NAT Mechanism

The computation flow of the proposed NAT is summarized in Algorithm 1. Since each layer is processed sequentially, we use the j^{th} layer as an example. The training flow of the j^{th} layer is depicted in Fig. 8. First, the floating-point input activations A_{j-1} are clipped within a range of $[\beta_j, \alpha_j]$ by E-PACT. The clipped activations $A_{j-1_{\text{clip}}}$ are transferred into the forward function, which consists of the following steps:

1. Split the activations and the weights into several tiles to fit the size of the memory crossbars, denoted as cs .
2. Quantize the activations and the weights with the configurations of the CIM macro, where the scaling factors, s_{weight} and $s_{\text{activation}}$, convert the weights and the activations between floating-point and fixed-point.
3. Perform matrix-vector multiplications (MVMs) with the quantized activations and weights to obtain partial sums, which are further truncated to fit the resolution of ADCs.
4. The ideal partial sums $A_{j_{\text{ideal}}}$ are transformed to the non-ideal measured values based on the D-MAC-EM.
5. The measured partial sums $A_{j_{\text{error}}}$ are aggregated and converted back to the floating-point numbers for the next layer.

After forward passing all the layers, the gradients of model weights and clipping ranges are computed by the statistical training mechanism and E-PACT, respectively. The details of the statistical training mechanism and E-PACT are elaborated in the following subsections.

B. Statistical Training Mechanism

1) *Challenges of Training With D-MAC-EM*: To mitigate the non-ideal effects of CIM, we train DNNs with the D-MAC-EM obtained from the measurement results of a practical CIM. One major challenge of training with a discrete function is that derivatives of the function are zero everywhere. This causes the gradients to be zero and prevents updating the DNN model. The technique of straight-through-estimator (STE) [24] is widely implemented in discrete optimization using stochastic gradient descent (SGD) due to its effectiveness and simplicity. STE estimates the gradients of a discrete function as an identity function, which means STE directly bypasses the incoming gradients to the preceding layer. However, when considering the non-ideal D-MAC-EM, the forward propagation is a non-linear curve with different probability distributions. Simply using STE to estimate the gradients during the back propagation degrades the effectiveness of the training process. Therefore, the gradients of training the model with such D-MAC-EM need to be re-design.

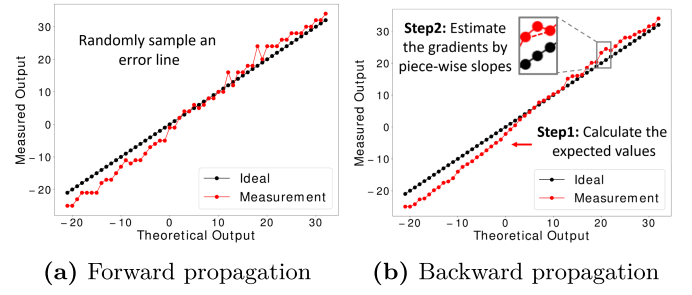


Fig. 9. The proposed statistical training flow. (a) Forward propagation: randomly sample an error line from the D-MAC-EM. (b) Backward propagation: compute the expected values for each point and calculate the gradients as the slope of the piece-wise linear curve between two adjacent points.

2) *Computation Flow of Statistical Training*: We propose a statistical training mechanism to correctly estimate the gradients of the D-MAC-EM. The D-MAC-EM, denoted as Err , can be considered as a discrete distribution that consists of $2^k - 1$ independent discrete random variable: $X_{-(2^{k-1}-1)}, \dots, X_{2^{k-1}-1}$, where k is the resolution of the ADC and X_i denotes all possible measured outputs for the theoretical output i . The values of i and X_i are integers within $[-(2^{k-1}-1), 2^{k-1}-1]$. For example, X_{21} represents the probability distribution of measured outputs corresponding to the theoretical output 21.

Fig. 9 illustrates the proposed training flow. For each MVM during the forward propagation, we randomly sample a value from each distribution to form an error line to represent the data-dependent behavior, as shown in Fig. 9(a). This error line is then applied to the outputs to simulate the non-ideal effects of the SRAM-based CIM. The procedure of the backward propagation is shown in Fig. 9(b). We first compute the expected values of each distribution to obtain an expected error line, where the expected value of X_i is denoted as $\mathbb{E}[X_i]$. The gradient of each point is estimated as the slope of the piece-wise linear curve between two adjacent points, which is formulated as:

$$\nabla_x Err|_{x=i} = \frac{\partial Err}{\partial x}|_{x=i} = \frac{\mathbb{E}[X_{i+1}] - \mathbb{E}[X_{i-1}]}{2}. \quad (2)$$

The calculated gradient for each output value is then used to update the model parameters.

C. Extended PACT (E-PACT)

1) *Trade-Offs of Quantization Range and Quantization Errors*: DNN models must be quantized to fit the configurations of the SRAM-based CIM macro. Since the weights in our design are 1-bit, we quantize the weights with a uniform quantization scheme and focus on quantizing input activations. A typical uniform quantization scheme is shown as follows:

$$I_q = \text{round}\left(\frac{I - z}{s}\right) \times s + z, \quad (3)$$

where I , I_q denote the floating-point value and quantized value, respectively, and $\text{round}(\cdot)$ maps the inputs to the nearest integer values. z is the offset of the quantization range, which is generally set as the minimum value of I . The scaling factor s is defined as:

$$s = \frac{\text{range}(I)}{2^k - 1}, \quad (4)$$

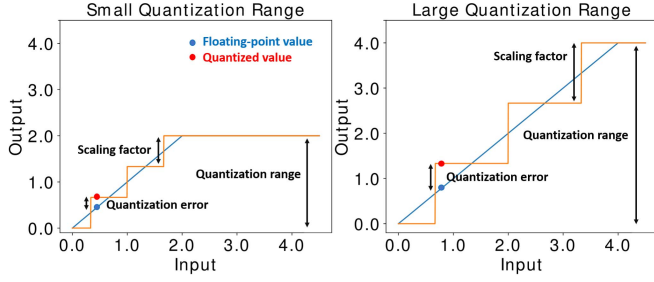


Fig. 10. Illustration of a typical linear quantization scheme. A larger quantization range causes larger quantization errors.

where $range(\cdot)$ represents the dynamic range of input activations, and k is the resolution of the DAC. The scaling factor can be regarded as the step size between each quantized level, as illustrated in Fig. 10. A broader dynamic range can cover more numbers, thus having a higher model capability. However, it also introduces a larger scaling factor which causes more deviations between floating-point values and their quantized values.

The design of the quantization range is often considered together with the activation function. ReLU is a commonly used activation function for DNNs, which is defined by:

$$ReLU(x) = \begin{cases} x, & x > 0 \\ 0, & \text{otherwise} \end{cases} \quad (5)$$

The dynamic range of the ReLU function is calculated as $max(I)$, which has an unbounded range. The unbounded quantization range may lead to a large quantization error. Therefore, Clipped ReLU [3] is often used to restrict the inputs to a specific range $[0, \alpha]$, which can be formulated as follows:

$$Clipped_ReLU(x, \alpha) = \begin{cases} \alpha, & x > \alpha \\ x, & x \in [0, \alpha] \\ 0, & \text{otherwise} \end{cases} \quad (6)$$

To verify the relationship between quantization range and quantization errors, we perform MAC operation with floating-point inputs and quantized inputs and calculate their differences under different ranges, as illustrated in 11(a). The results are shown in Fig. 11(b). The computation errors are more significant when the input has a wider dynamic range. To further investigate the impact of the quantization errors on the model accuracy, we apply the clipped ReLU with the range of $[0, \alpha]$ to ResNet20 under different training scenarios. The FP32 model is trained with floating-point values. The Ideal CIM model is trained with quantized values considering the quantization errors. The Non-Ideal CIM model is trained with quantized values and the D-MAC-EM, which simultaneously considers the quantization errors and non-ideal effects of CIM. Fig. 12 illustrates the results. As the quantization range increases, the accuracy of the FP32 model monotonically increases due to higher model capability. As for the Ideal CIM model, the accuracy decreases when the quantization range exceeds 5 due to larger quantization errors. Moreover, considering the D-MAC-EM, the accuracy of the Non-Ideal CIM model drastically degrades when the quantization range

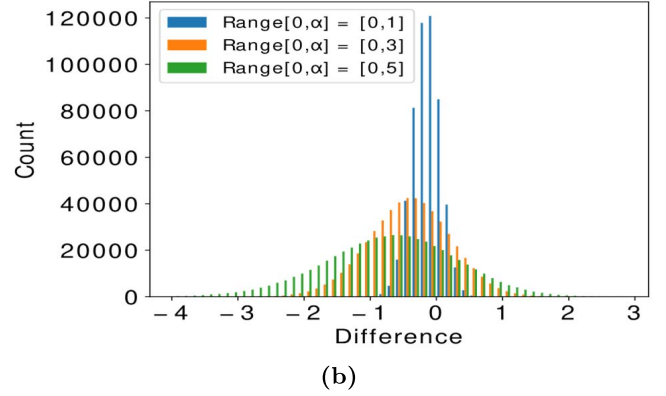
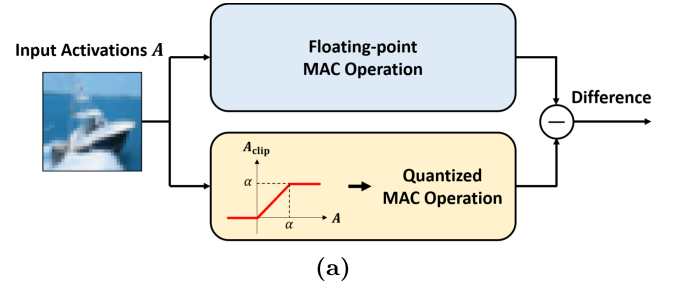


Fig. 11. (a) Calculation of the quantization errors after MAC operation. (b) Quantization errors under different quantization ranges $[0, \alpha]$.

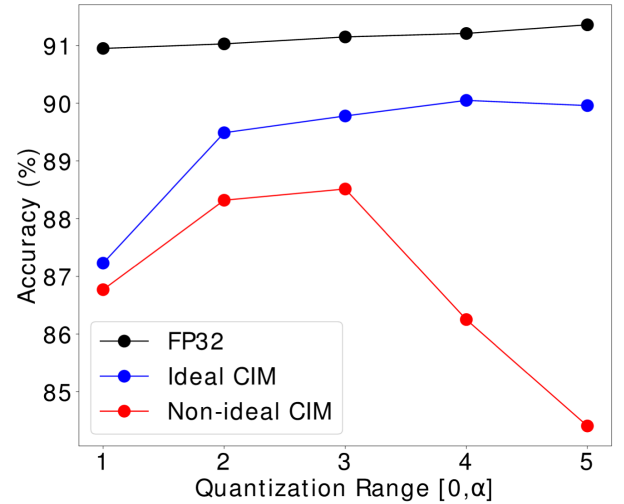


Fig. 12. The trade-offs between quantization range and quantization errors under the CIM non-idealities.

is larger than 3. Based on the experimental results, selecting a suitable quantization range can effectively mitigate the non-ideal effects ($\alpha = 3$ in this case). Therefore, the quantization range should be carefully determined to achieve high accuracy for the non-ideal CIM.

2) *Proposed E-PACT Algorithm*: To find a suitable quantization range for quantized models, PACT [25] parameterizes the upper bound α for each layer and adaptively updates them during training. However, PACT assumes that the quantization ranges are unsigned where all values below zero are discarded, which leads to information loss. Therefore, we introduce E-PACT to automatically determine the suitable quantization ranges, including lower and upper bounds for each layer under

TABLE I
ACCURACY OF DIFFERENT GRADIENT ESTIMATORS

Gradient Estimator	CIFAR-10	
	ResNet20	VGG8
STE [24]	86.15%	87.45%
Proposed Statistical Training	87.45% (+1.3%)	88% (+0.55%)

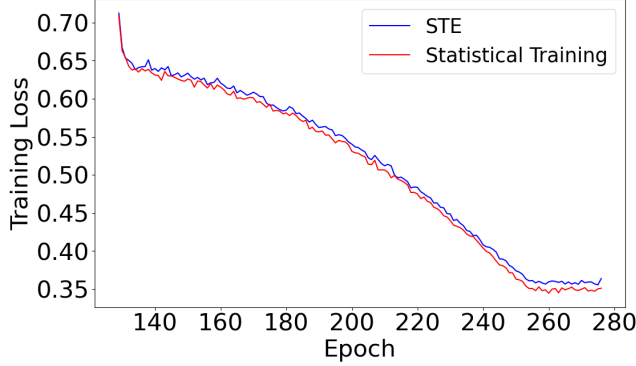


Fig. 13. Comparison of the convergence performance between the proposed statistical training and STE.

the non-ideal SRAM-based CIM macro. E-PACT parameterizes the upper and lower bound for the quantization range as α and β and clips the input activations A to A_{clip} , which is formulated as follows:

$$A_{clip} = E-PACT(A, \alpha, \beta) \begin{cases} \alpha, & A \in [\alpha, \infty) \\ A, & A \in (\beta, \alpha) \\ \beta, & A \in (-\infty, \beta] \end{cases} \quad (7)$$

After each layer finishes the forward pass, we compute the gradients of α and β as:

$$\frac{\partial A_{clip}}{\partial \alpha} = \begin{cases} 1, & A > \alpha \\ 0, & A \leq \alpha \end{cases}, \quad (8)$$

$$\frac{\partial A_{clip}}{\partial \beta} = \begin{cases} 0, & A \geq \beta \\ 1, & A < \beta \end{cases}. \quad (9)$$

The calculated gradients are then used to update α and β .

V. SIMULATION RESULTS

A. Dataset and Simulation Settings

We implement our proposed NAT mechanism with ResNet20 and VGG8 on the CIFAR-10 [27] dataset. The hardware settings are the same as the SRAM-based CIM macro [1] with 7-bit input activations, 1-bit weights, and 7-bit partial sums. The D-MAC-EM and A-MAC-EM we used are explained in Section III. For each training process, we set the initial learning rate as 0.1 with cyclic cosine-annealing scheduler, the number of epochs is 256, and the batch size is set as 128. We use stochastic gradient descent (SGD) as our optimizer and set the weight decay as 0.0005. All the experiments are operated with PyTorch1.9.0 and Python3.9.

TABLE II
ACCURACY OF DIFFERENT QUANTIZATION SCHEMES

Quantization Scheme	CIFAR-10	
	ResNet20	VGG8
ReLU	58.07%	76.23%
Clipped ReLU [3]	86.15%	87.45%
PACT [25]	87.07%	87.78%
Proposed E-PACT	88.17% (+1.1%)	88.29% (+0.51%)

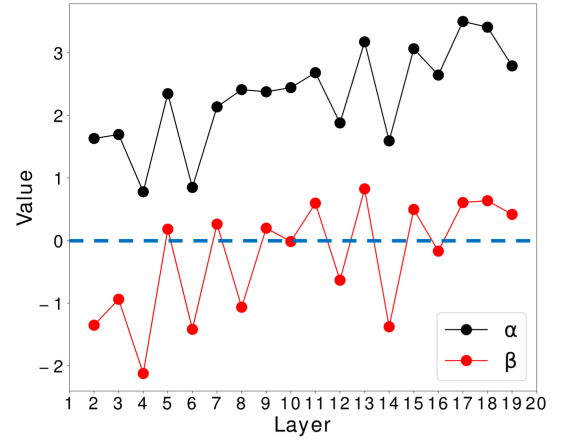


Fig. 14. Dynamic range of each layer in ResNet20 found by E-PACT.

B. Analysis of Statistical Training

In this part, we compare the performance of the statistical training mechanism with STE. Both methods use clipped ReLU as the quantization scheme and are trained with the D-MAC-EM for a fair comparison. Table I shows the accuracy of ResNet20 and VGG8 under different scenarios. The statistical training improves the accuracy of ResNet20 and VGG8 by 0.55% and 1.3% compared with STE owing to the optimization of calculating the gradients. Moreover, Fig. 13 shows the training loss during the training process. We can observe that statistical training leads STE by 5 epochs and achieves a lower training loss by correctly estimating the gradients of the D-MAC-EM.

C. Analysis of Different Quantization Schemes

To validate the effectiveness of the proposed E-PACT, we compare E-PACT with other quantization schemes. For a fair comparison, all the methods use STE as the gradient estimator and are trained with the D-MAC-EM. Table II shows the simulation results under different quantization schemes. As expected, E-PACT has the highest accuracy because the best quantization range is learned for each layer to find the best trade-offs between quantization range and quantization errors.

Fig. 14 illustrates the upper and lower bounds of each layer of ResNet20 after training. We can observe that about half

TABLE III
EVALUATION OF THE PROPOSED NAT MECHANISM ON SMALL-SCALE AND LARGE-SCALE FRAMEWORKS

Framework	CIFAR-10		CIFAR-100	
	ResNet20	VGG8	ResNet34	VGG16
FP32	92.6%	91.18%	79.84%	72.87%
Ideal CIM	89.5%	87.67%	78.07%	68.94%
Non-Ideal CIM	10.14%	16.78%	1.15%	1.13%
A-NAT	13.88%	20.39%	8.82%	6.56%
Proposed D-NAT	89.12% (+78.98%)	88.58% (+71.8%)	73.19% (+72.04%)	58.98% (+57.85%)

of the layers tend to learn a lower negative bound, which means that the information can be retained to achieve higher accuracy by considering the negative values. Compared with other quantization schemes, E-PACT can improve the accuracy of ResNet20 and VGG8 by 1.1% ~ 30.1% and 0.51% ~ 12.06%, respectively.

D. Comparison of Overall Performance

Finally, we evaluate the effectiveness of the A-MAC-EM and D-MAC-EM based on our proposed NAT mechanism, as listed in Table III. To evaluate the scalability of the framework, we conduct more experiments on a larger dataset and models, including ResNet34 and VGG16 on CIFAR-100. The upper three cases are trained without considering the MAC error model. The FP32 model is trained and tested with floating-point numbers, which shows the highest accuracy. The Ideal CIM and Non-Ideal CIM are trained with quantized numbers that correspond to the hardware settings of the SRAM-based CIM macro (input activation: 7b, weight: 1b). The Non-Ideal CIM is tested with the D-MAC-EM. On the contrary, the Ideal-CIM is tested without considering the non-ideality of the SRAM-based CIM macro, which is the ideal upper bound of the quantized model. The accuracy drop between FP32 and Ideal CIM is mainly due to the quantization errors. When non-ideal effects are further considered (Non-Ideal CIM case), computation errors tend to propagate and accumulate across each layer. Especially in large-scale frameworks (ResNet34 and VGG16 on CIFAR-100), the models are deeper (more accumulated errors) and the classification tasks become more difficult. Therefore, the computation errors grow substantially worse, leading to significant accuracy degradation.

For the non-ideality-aware training cases, the A-NAT is trained with the A-MAC-EM using STE and Clipped ReLU, and the D-NAT is trained with the D-MAC-EM using statistical training and E-PACT. Both of the cases are tested with the D-MAC-EM. The experiment results show that the A-NAT slightly improves the accuracy of the models; however, it still remains a low accuracy. On the other hand, D-NAT successfully recovers the accuracy of ResNet20, VGG8, ResNet34, and VGG16 by 78.98%, 71.8%, 72.04%, and 57.85%, respectively, which reaches the ideal upper bound of the quantized model by further considering the information in negative activations.

TABLE IV
MEASUREMENT RESULTS OF ON-CHIP VALIDATION

Software Simulation	
FP32	546/600 (90.1%)
Ideal CIM	530/600 (88.33%)
On-Chip Validation	
Ideal pretrained model	85/600 (14.17%)
D-NAT trained model	523/600 (87.17%)

In summary, the A-MAC-EM is not sufficient to simulate the practical non-idealities in an SRAM-based CIM macro. Therefore, measured statistics are required for NAT to achieve high accuracy.

VI. VALIDATION OF D-NAT ON THE FABRICATED SRAM-BASED CIM MACRO

In this section, we validate the proposed D-NAT framework on the fabricated SRAM-based CIM macro. The DNN model we use is VGG8 with the same hardware settings as mentioned in Section III. Since the size of on-chip buffers is limited, we select 600 images from the CIFAR-10 dataset. To ensure the selected images can better represent the whole dataset, we select the images evenly from each class, and the accuracy of the selected images is similar to that of the whole dataset in software simulations. For software simulation, the model of FP32 and Ideal CIM is the same as the ones in Table III. For on-chip validation, we deploy the Ideal CIM trained model and the D-NAT trained model on the fabricated SRAM-based CIM macro for evaluation. Specifically, we compute the first convolution layer on the SRAM-based CIM macro and compute the remaining layers with the D-MAC-EM.

Table IV shows the number of correctly predicted images out of the selected 600 images. For the results of software simulation, the accuracy of FP32 and Ideal CIM achieve 90.1% and 88.33%, respectively. The results show that directly computing the Ideal CIM trained model with the SRAM-based CIM macro severely degrades the accuracy by 74.16%. On the other hand, by incorporating the D-MAC-EM into the training process, our proposed D-NAT effectively improves the

accuracy from 14.17% to 87.17% under the non-idealities of a real SRAM-based CIM macro.

Currently, our proposed training method induces extra training overhead compared with the normal FP32 training method, which is about 5x training time and 2x GPU memory usage based on our experiments. However, those training efforts are done offline with one-time effort. Then, it can be applied to on-chip inference with high accuracy, as illustrated in Table IV.

VII. CONCLUSION

In this paper, we present a data-driven non-ideality aware training (D-NAT) framework to compensate for the non-ideal effects of analog computing in CIM. We observe the deficiency of the analytical MAC error model (A-MAC-EM) compared with the data-driven MAC error model (D-MAC-EM). The proposed statistical training mechanism shows a more stable and faster convergence speed with 0.55% and 1.3% accuracy improvement on ResNet20 and VGG8 compared with STE. The proposed E-PACT provides a 1.1% $\sim$ 30.1% and 0.51% $\sim$ 12.06% accuracy improvement on ResNet20 and VGG8 compared with other quantization methods. The overall simulation results show that our proposed D-NAT improves the accuracy of ResNet20, VGG8, ResNet34, and VGG16 by 78.98%, 71.8%, 72.04%, and 57.85%, respectively. Finally, we validate the D-NAT trained model on the fabricated SRAM-based CIM chip. The measurement results show that our proposed D-NAT successfully recovers the accuracy of VGG8 from 14.17% to 87.17% without additional hardware circuits.

The proposed D-NAT mechanism can serve as the reference design for other types of CIM macros based on emerging memories, as long as they have enough measurement results of the fabricated CIM macros.

REFERENCES

- [1] Y.-T. Hsu, C.-Y. Yao, T.-Y. Wu, T.-D. Chiueh, and T.-T. Liu, "A high-throughput energy-area-efficient computing-in-memory SRAM using unified charge-processing network," *IEEE Solid-State Circuits Lett.*, vol. 4, pp. 146–149, 2021.
- [2] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, vol. 25, Stateline, NV, USA, Dec. 2012, pp. 1097–1105.
- [3] D. Amodei *et al.*, "Deep speech 2: End-to-end speech recognition in english and Mandarin," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 173–182.
- [4] Y. Wu *et al.*, "Google's neural machine translation system: Bridging the gap between human and machine translation," 2016, *arXiv:1609.08144*.
- [5] A. Sebastian *et al.*, "Memory devices and applications for in-memory computing," *Nat. Nanotechnol.*, vol. 15, pp. 529–544, Jul. 2020.
- [6] N. Verma *et al.*, "In-memory computing: Advances and prospects," *IEEE Solid State Circuits Mag.*, vol. 11, no. 3, pp. 43–55, Aug. 2019.
- [7] S. Yu, X. Sun, X. Peng, and S. Huang, "Compute-in-memory with emerging nonvolatile-memories: Challenges and prospects," in *Proc. IEEE Custom Integr. Circuits Conf. (CICC)*, Mar. 2020, pp. 1–4.
- [8] X. Sun and S. Yu, "Impact of non-ideal characteristics of resistive synaptic devices on implementing convolutional neural networks," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 9, no. 3, pp. 570–579, Sep. 2019.
- [9] Y. Long, X. She, and S. Mukhopadhyay, "Design of reliable DNN accelerator with un-reliable ReRAM," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Mar. 2019, pp. 1769–1774.
- [10] C.-J. Jhang, C.-X. Xue, J.-M. Hung, F.-C. Chang, and M.-F. Chang, "Challenges and trends of SRAM-based computing-in-memory for AI edge devices," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 68, no. 5, pp. 1773–1786, Mar. 2021.
- [11] S. Zhang, K. Huang, and H. Shen, "A robust 8-bit non-volatile computing-in-memory core for low-power parallel MAC operations," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 67, no. 6, pp. 1867–1880, Jun. 2020.
- [12] B. Yan *et al.*, "Resistive memory-based in-memory computing: From device and large-scale integration system perspectives," *Adv. Intell. Syst.*, vol. 1, no. 7, 2019, Art. no. 1900068.
- [13] H. Kim, Q. Chen, and B. Kim, "A 16K SRAM-based mixed-signal in-memory computing macro featuring voltage-mode accumulator and row-by-row ADC," in *Proc. IEEE Asian Solid-State Circuits Conf. (A-SSCC)*, Nov. 2019, pp. 35–36.
- [14] S. Yin, Z. Jiang, J.-S. Seo, and M. Seok, "XNOR-SRAM: In-memory computing SRAM macro for binary/ternary deep neural networks," *IEEE J. Solid-State Circuits*, vol. 55, no. 6, pp. 1733–1743, Jun. 2020.
- [15] Q. Dong *et al.*, "15.3 A 351TOPS/W and 372.4GOPS compute-in-memory SRAM macro in 7nm FinFET CMOS for machine-learning applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2020, pp. 242–244.
- [16] C. Yu, T. Yoo, H. Kim, T. T.-H. Kim, K. C. T. Chuan, and B. Kim, "A logic-compatible eDRAM compute-in-memory with embedded ADCs for processing neural networks," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 68, no. 2, pp. 667–679, Feb. 2020.
- [17] Z. Lin *et al.*, "Cascade current mirror to improve linearity and consistency in SRAM in-memory computing," *IEEE J. Solid-State Circuits*, vol. 56, no. 8, pp. 2550–2562, Aug. 2021.
- [18] S. Jain and A. Raghunathan, "CxENN: Hardware-software compensation methods for deep neural networks on resistive crossbar systems," *ACM Trans. Embedded Comput. Syst.*, vol. 18, no. 6, pp. 1–23, 2019.
- [19] B. Zhang, L.-Y. Chen, and N. Verma, "Stochastic data-driven hardware resilience to efficiently train inference models for stochastic hardware implementations," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, May 2019, pp. 1388–1392.
- [20] A. Bhattacharjee, L. Bhatnagar, Y. Kim, and P. Panda, "NEAT: Non-linearity aware training for accurate, energy-efficient and robust implementation of neural networks on 1T-1R crossbars," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, early access, Sep. 3, 2021, doi: [10.1109/TCAD.2021.3109857](https://doi.org/10.1109/TCAD.2021.3109857).
- [21] B. Jacob *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 2704–2713.
- [22] Z. Jiang, S. Yin, J.-S. Seo, and M. Seok, "C3SRAM: An in-memory-computing SRAM macro based on robust capacitive coupling computing mechanism," *IEEE J. Solid-State Circuits*, vol. 55, no. 7, pp. 1888–1897, Jul. 2020.
- [23] J. Zhang, Z. Wang, and N. Verma, "In-memory computation of a machine-learning classifier in a standard 6T SRAM array," *IEEE J. Solid-State Circuits*, vol. 52, no. 4, pp. 915–924, Apr. 2017.
- [24] Y. Bengio, N. Léonard, and A. Courville, "Estimating or propagating gradients through stochastic neurons for conditional computation," 2013, *arXiv:1308.3432*.
- [25] J. Choi, Z. Wang, S. Venkataramani, P. I.-J. Chuang, V. Srinivasan, and K. Gopalakrishnan, "Pact: Parameterized clipping activation for quantized neural networks," 2018, *arXiv:1805.06085*.
- [26] V. Nair and G. E. Hinton, "Rectified linear units improve restricted Boltzmann machines," in *Proc. ICML*, 2010, pp. 1–8.
- [27] A. Krizhevsky, *et al.*, "Learning multiple layers of features from tiny images," Univ. Toronto, Toronto, ON, Canada, Tech. Rep., 2009.
- [28] S. Yu, H. Jiang, S. Huang, X. Peng, and A. Lu, "Compute-in-memory chips for deep learning: Recent trends and prospects," *IEEE Circuits Syst. Mag.*, vol. 21, no. 3, pp. 31–56, 3rd Quart., 2021.
- [29] C.-C. Liu, S.-J. Chang, G.-Y. Huang, and Y.-Z. Lin, "A 10-bit 50-MS/s SAR ADC with a monotonic capacitor switching procedure," *IEEE J. Solid-State Circuits*, vol. 45, no. 4, pp. 731–740, Apr. 2010.
- [30] C.-P. Huang, H.-W. Ting, and S.-J. Chang, "Analysis of non-ideal behaviors based on INL/DNL plots for SAR ADCs," *IEEE Trans. Instrum. Meas.*, vol. 65, no. 8, pp. 1804–1817, Aug. 2016.



Ming-Guang Lin received the B.S. degree in electrical engineering from the National Taiwan University, Taipei, Taiwan, in 2020, where he is currently pursuing the Ph.D. degree in electronics engineering. His research interests include in-memory-computing-based systems, machine learning algorithms, and VLSI implementation.



Jyun-Jhe Chou received the B.S. degree in computer science and information engineering from the National Taiwan University in 2014, where he is currently pursuing the Ph.D. degree with the Department of Computer Science and Information Engineering. His research interests include the Internet of Things, quality of data, infrared image processing, and cyber-physical systems.



Chi-Tse Huang received the B.S. degree in electrical engineering from the National Taiwan University, Taipei, Taiwan, in 2021, where he is currently pursuing the Ph.D. degree in electronics engineering. His research interests include deep learning applications, in-memory-computing, and variation-aware training.



Tsung-Te Liu (Member, IEEE) received the B.S. and M.S. degrees in electrical engineering from the National Taiwan University, Taiwan, in 2002 and 2004, respectively, and the Ph.D. degree in electrical engineering from the University of California at Berkeley in 2012. From 2004 to 2005, he was with MediaTek Inc., Taiwan, where he was involved in circuit and system design for wireless communications. From 2005 to 2012, he was a member of the Berkeley Wireless Research Center (BWRC), University of California at Berkeley. From 2012 to 2014,

he was with the Interuniversity Microelectronics Centre (IMEC), Belgium, where he conducted research on circuit development for advanced CMOS technology. In 2014, he joined the Faculty of the National Taiwan University, where he is currently an Associate Professor with the Graduate Institute of Electronics Engineering and the Department of Electrical Engineering. His research interests involve energy-efficient circuit and system designs. He was a recipient of several design and teaching awards.



Yu-Chuan Chuang received the B.S. degree in electronic engineering from the National Taiwan University, Taipei, Taiwan, in 2018, where he is currently pursuing the Ph.D. degree with the Graduate Institute of Electronics Engineering. His research interests include low-power design, hyperdimensional computing, and VLSI implementation.



Chi-Sheng Shih (Senior Member, IEEE) received the B.S. degree in engineering science and the M.S. degree in computer science from the National Cheng Kung University in 1993 and 1995, respectively, and the Ph.D. degree in computer science from the University of Illinois at Urbana-Champaign in 2003. He is currently a Professor at the Graduate Institute of Networking and Multimedia and the Department of Computer Science and Information Engineering, National Taiwan University. He works as the Director of GINM and the Interim Director of the High Performance Scientific Computing Technology Center. His main research interests include embedded systems, hardware/software codesign, real-time systems, middleware, and cyber-physical systems. His research results won several awards, including the Distinguished Electronic Engineering Professor Award 2021 and the Best Paper Award in 2019 MobiSec/Future ICT, 2016 IEEE International Symposium on Cloud and Service Computing, 2011 ACM Research in Applied Computation Symposium (RACS 2011), IEEE RTCSA 2005, and IEEE RTSS 2004. He leads the NTU RoboPAL Team, which won the First Place of Shield Challenge of Standard Platform League in RoboCup 2018. He serves as an Associate Editor for *IET Cyber-Physical Systems: Theory and Applications* and *Service Oriented Computing and Applications* (SOCA, Springer).



Yi-Ta Chen (Student Member, IEEE) received the B.S. degree in electrical engineering from the National Taiwan University, Taipei, Taiwan, in 2017, where he is currently pursuing the Ph.D. degree with the Graduate Institute of Electronics Engineering. His research interests include the machine learning engine for affective computing, biosignal processing and feature extraction for affective computing, and SW/HW co-design for SDN data plane.



Ying-Tuan Hsu received the B.S. and M.S. degrees in electrical engineering from the National Taiwan University, Taipei, Taiwan, in 2018 and 2021, respectively. He is currently a Research Assistant with the EECS Laboratory, National Taiwan University. His research interests involve mixed-signal circuit and energy-efficient circuit designs.



An-Yeu (Andy) Wu (Fellow, IEEE) received the B.S. degree in electrical engineering from the National Taiwan University (NTU), Taipei, Taiwan, in 1987, and the M.S. and Ph.D. degrees in electrical engineering from the University of Maryland of College Park, MD, USA, in 1992 and 1995, respectively. In 2000, he joined the Department of Electrical Engineering and the Graduate Institute of Electronics Engineering (GIEE), NTU, as a Faculty Member, where he is currently a Distinguished Professor and worked as the Director of GIEE from 2016 to 2019.

From 2007 to 2009, he was on leave from NTU and worked as the Deputy General Director of the SoC Technology Center (STC), Industrial Technology Research Institute (ITRI), Hsinchu, Taiwan. His research interests include VLSI architectures for signal processing and communications and adaptive/multi-rate signal processing. He has published more than 250 refereed journals and conference papers in the above research areas, together with five book chapters and 20 granted U.S. patents. He was elevated to IEEE Fellow for his contributions to DSP algorithms and VLSI designs for communication IC/SoC in 2015.



Yu-Kai Chen received the B.S. degree in electrical engineering from the National Chung Hsing University, Taichung, Taiwan, in 2021. He is currently pursuing the M.S. degree with the Graduate Institute of Electronics Engineering, National Taiwan University, Taiwan. His research interests involve computing-in-memory and mixed-signal circuit designs for machine learning applications.